

Module 10: Log-Based Recovery & WAL Protocol

Log-Based Recovery & Write-Ahead Logging (WAL)

Log Record Anatomy, WAL Invariant, Deferred vs Immediate Modification & Undo/Redo Logic

The Write-Ahead Logging (WAL) Sacred Protocol:

- Before database page on disk is updated with new value V_{new} , the log record MUST be written to stable storage!
- Before $\langle Ti \text{ commit} \rangle$ log record is confirmed to user, ALL preceding log records for Ti MUST be flushed to disk.

Guarantee: If crash occurs mid-flight, recovery manager always has log history to Undo or Redo!

1. Deferred Database Modification

"No-Undo / Redo-Only" Recovery Policy

Mechanism:

- Physical disk updates are POSTPONED until AFTER commit!
- While active, writes are recorded ONLY in log file and memory.

Log records only need NEW value: $\langle Ti, X, V_{new} \rangle$

Crash Recovery Action:

- If log has $\langle Ti \text{ start} \rangle$ and $\langle Ti \text{ commit} \rangle \Rightarrow \text{REDO}(Ti)$.
- If log has $\langle Ti \text{ start} \rangle$ but NO commit $\Rightarrow$ NO ACTION NEEDED!
(Since uncommitted data was never written to disk!).

✓ Zero Undo overhead! ✗ Requires large memory buffers.

2. Immediate Database Modification

"Undo / Redo" Standard Enterprise Policy

Mechanism:

- Database disk can be updated while transaction is ACTIVE.
- Disk may contain uncommitted "dirty" updates at any time.

Log records MUST have BOTH values: $\langle Ti, X, V_{old}, V_{new} \rangle$

Crash Recovery Action:

- If log has $\langle Ti \text{ start} \rangle$ and $\langle Ti \text{ commit} \rangle \Rightarrow \text{REDO}(Ti)$.
- If log has $\langle Ti \text{ start} \rangle$ but NO commit $\Rightarrow \text{UNDO}(Ti)$!
(Restores $X = V_{old}$ to eliminate dirty uncommitted changes).

Idempotency Guarantee: $\text{Redo}(\text{Redo}) = \text{Redo}$, $\text{Undo}(\text{Undo}) = \text{Undo}$.

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Log Record Architecture, Write-Ahead Logging (WAL) Protocol, Deferred Database Modification vs Immediate Database Modification, Undo and Redo Algorithms, Idempotency.

1. What is a Database Log?

Database **Log** stable storage par likha gaya ek append-only audit trail hota hai jisme transaction execution ke har operation ka chronological record rehta hai.

Har log record ka structure:

- $\langle T_i \text{ ext} \{ \text{start} \} \rangle$

angle\$: Transaction T_i shuru hua.

- $\langle T_i, X, V_{old}, V_{new} \rangle$

angle\$: T_i ne data item X ko purani value V_{old} (before-image) se badal kar nayi value V_{new} (after-image) kar diya.

- $\langle T_i \text{ ext} \{ \text{commit} \} \rangle$

angle\$: T_i successfully commit hua.

- $\langle T_i \text{ ext} \{ \text{abort} \} \rangle$

angle\$: T_i abort hua aur rollback complete hua.

2. Write-Ahead Logging (WAL) Protocol

Crash recovery ki durability aur atomicity ensure karne ke liye WAL protocol follow karna compulsory hai:

WAL Rule 1: Disk par data page ko new value se overwrite karne se pehle, us update ka log record non-volatile stable storage par flush ho chuka hona chahiye!

WAL Rule 2: User ko commit acknowledgment bhejne se pehle, us transaction ke saare log records stable storage par physically written hone chahiye!

3. Deferred vs Immediate Database Modification

Parameter	Deferred Modification (NO UNDO)	Immediate Modification (UNDO & REDO)
When are Disk Pages Updated?	Only AFTER commit	Immediately while active
Log Record Contents		

Both old & new: $\langle T_i, X, V_{old}, V_{new} \rangle$

Crash Action for Active Uncommitted	No action! (Disk was never touched)	UNDO (Restore $X = V_{old}$)
Crash Action for Committed	REDO ($X = V_{new}$)	REDO ($X = V_{new}$)
Memory Buffer Pressure	High (must hold all updates in RAM)	Low (can flush dirty pages anytime)
Industry Adoption	Rarely used in high-volume systems	Standard in Oracle, MySQL, SQL Server

4. Undo and Redo Operation Primitives

Undo(T_i): Log ko backward scan karke har update record $\langle T_i, X, V_{old}, V_{new} \rangle$

ke liye data item X ko uski purani value V_{old} set karta hai.

Redo(T_i): Log ko forward scan karke har update record $\langle T_i, X, V_{old}, V_{new} \rangle$

ke liye data item X ko uski nayi value V_{new} set karta hai.

Idempotence Property: Agar recovery process ke dauran dubara system crash ho jaye, toh Undo aur Redo ko kitni bhi baar repeat run kiya ja sakta hai bina koi galat outcome generate kiye!

$$\text{Redo}(\text{Redo}(T_i)) = \text{Redo}(T_i)$$

5. Architectural Diagram